

1. Diseño de URL (Endpoints)

✓ Usa nombres de recursos (sustantivos), no acciones (verbos)

✗ GET /getUsers

✓ GET /users

✓ Usa nombres en plural

✗ GET /user/1

✓ GET /users/1

✓ Usa subrecursos para relaciones

✗ GET /orders?userId=1

✓ GET /users/1/orders

✓ Evita incluir extensiones de archivos

✗ GET /users.json

✓ GET /users

✓ Utiliza guiones (-) en vez de guion bajo (_)

✗ GET /user_orders

✓ GET /user-orders

2. Métodos HTTP adecuados

Método	Propósito
GET	Obtener datos
POST	Crear un recurso
PUT	Actualizar completamente un recurso
PATCH	Actualizar parcialmente un recurso
DELETE	Eliminar un recurso

Ejemplo:

- ✓ POST /users → Crear usuario
- ✓ PUT /users/1 → Reemplazar usuario
- ✓ PATCH /users/1 → Actualizar email del usuario
- ✓ DELETE /users/1 → Eliminar usuario

3. Códigos de estado HTTP apropiados

Código	Significado
200 OK	Respuesta exitosa
201 Created	Recurso creado
204 No Content	Operación exitosa sin respuesta
400 Bad Request	Petición malformada
401 Unauthorized	No autenticado
403 Forbidden	Sin permisos
404 Not Found	Recurso no encontrado
409 Conflict	Conflicto con el estado actual
500 Internal Server Error	Error inesperado

4. Parámetros y Query Strings

- ✓ Usa parámetros de ruta para identificar recursos

```
GET /users/1
```

- ✓ Usa query strings para filtros y paginación

```
GET /users?role=admin&page=2&limit=10
```

5. Seguridad

- ✓ Usa HTTPS en todas las solicitudes
- ✓ Autenticación con tokens JWT o OAuth2
- ✓ No expongas claves de API en URLs
- ✗ GET /users?api_key=123456
- ✓ Usa encabezados: Authorization: Bearer <token>
- ✓ Valida y escapa entradas de usuario para prevenir inyección SQL/XSS
- ✓ Usa rate limiting para evitar ataques DDoS
- ✓ Implementa CORS correctamente para controlar accesos desde otros dominios

6. Versionado

✓ Incluye versión en la URL o encabezado

✗ GET /users (sin versión)

✓ GET /v1/users

✓ GET /users con Accept: application/vnd.company.v1+json

7. Respuestas y Formato JSON

- ✓ Usa JSON como formato de respuesta
- ✓ Incluye solo la información necesaria
- ✓ Nombres en `camelCase` para los atributos

✗ `{"user_id": 1, "full_name": "John Doe"}`

✓ `{"userId": 1, "fullName": "John Doe"}`

- ✓ Estructura clara

json

 Copiar  Editar

```
{
  "data": {
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
  },
  "meta": {
    "timestamp": "2025-03-11T10:00:00Z"
  }
}
```



Manejo de errores consistente

json

 Copiar

 Editar

```
{  
  "error": {  
    "code": 404,  
    "message": "User not found"  
  }  
}
```


8. Paginación

✓ Usa parámetros estándar

```
GET /users?page=2&limit=20
```

Ejemplo de respuesta paginada:

json

 Copiar  Editar

```
{
  "data": [...],
  "pagination": {
    "currentPage": 2,
    "totalPages": 5,
    "totalItems": 100
  }
}
```

9. Caché

✓ Usa encabezados HTTP adecuados

- `Cache-Control: max-age=3600, public`
- `ETag` para verificar si un recurso cambió

✓ Evita recalcular datos pesados en cada solicitud

10. Documentación

- ✓ Usa herramientas como Swagger, Postman o Redoc
- ✓ Incluye ejemplos de uso y códigos de respuesta

Ejemplo con Swagger:

yaml

 Copiar  Editar

```
paths:
  /users:
    get:
      summary: "Obtener lista de usuarios"
      responses:
        200:
          description: "Lista de usuarios obtenida correctamente"
```